

TANGRAM: 一个基于比特切片的适合多平台的分组密码*

张文涛^{1,2}, 季福磊^{1,2}, 丁天佑^{1,2}, 杨博翰³, 赵雪锋^{1,2}, 向泽军⁴, 包珍珍⁵, 刘雷波³

1. 中国科学院 信息工程研究所 信息安全国家重点实验室, 北京 100093
2. 中国科学院大学 网络空间安全学院, 北京 100049
3. 清华大学 微电子学研究所, 北京 100084
4. 湖北大学 数学与统计学学院 应用数学湖北省重点实验室, 武汉 430062
5. 南洋理工大学, 新加坡 637371

通信作者: 张文涛, E-mail: zhangwentao@iie.ac.cn

摘要: 本文提出一族新的分组密码算法 TANGRAM. TANGRAM 包含三个版本: TANGRAM 128/128, 分组长度和密钥长度均为 128 比特; TANGRAM 128/256, 分组长度为 128 比特, 密钥长度为 256 比特; TANGRAM 256/256, 分组长度和密钥长度均为 256 比特. TANGRAM 分组密码采用 SP 网络, 我们对其 S 盒的选取以及线性层移位参数的选取进行了深入研究, 以使 TANGRAM 尽可能达到最优的安全性和实现性能的性价比. 我们深入分析了 TANGRAM 针对差分、线性、不可能差分、积分、相关密钥等重要密码分析方法的安全性, 为它预留了足够的安全冗余. 得益于比特切片方法, TANGRAM 在多种软件和硬件平台上都具有很好的表现, 可以灵活地适用于多种应用场景.

关键词: 分组密码; 比特切片方法; 安全性分析; 软件实现; 硬件实现; 侧信道防护

中图分类号: TP309.7 **文献标识码:** A **DOI:** 10.13868/j.cnki.jcr.000337

中文引用格式: 张文涛, 季福磊, 丁天佑, 杨博翰, 赵雪锋, 向泽军, 包珍珍, 刘雷波. TANGRAM: 一个基于比特切片的适合多平台的分组密码[J]. 密码学报, 2019, 6(6): 727-747.

英文引用格式: ZHANG W T, JI F L, DING T Y, YANG B H, ZHAO X F, XIANG Z J, BAO Z Z, LIU L B. TANGRAM: A bit-slice block cipher suitable for multiple platforms[J]. Journal of Cryptologic Research, 2019, 6(6): 727-747.

TANGRAM: A Bit-slice Block Cipher Suitable for Multiple Platforms

ZHANG Wen-Tao^{1,2}, JI Fu-Lei^{1,2}, DING Tian-You^{1,2}, YANG Bo-Han³, ZHAO Xue-Feng^{1,2}, XIANG Ze-Jun⁴, BAO Zhen-Zhen⁵, LIU Lei-Bo³

1. State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, Beijing 100093, China

* 基金项目: 国家自然科学基金 (61379138); IOT 安全联合研究项目 (Y8HX376678)

Foundation: National Natural Science Foundation of China (61379138); Joint Research Program for IOT Security (Y8HX376678)

收稿日期: 2019-11-22 定稿日期: 2019-12-10

2. School of Cyber Security, University of Chinese Academy of Sciences, Beijing 100049, China

3. Institute of Microelectronics, Tsinghua University, Beijing 100084, China

4. Hubei Key Laboratory of Applied Mathematics, Faculty of Mathematics and Statistics, Hubei University, Wuhan 430062, China

5. Nanyang Technological University, Singapore 637371, Singapore

Corresponding author: ZHANG Wen-Tao, E-mail: zhangwentao@iie.ac.cn

Abstract: This study proposes a new block cipher named TANGRAM. It has 3 different versions: TANGRAM 128/128, with block length and key length both being 128 bits; TANGRAM 128/256, with 128-bit block length and 256-bit key length; TANGRAM 256/256, with block length and key length both being 256 bits. TANGRAM uses an SP-network. To achieve the best possible security-performance tradeoff, a deep study has been made on the selection of the TANGRAM S-box and the rotation parameters of the linear layer. An extensive and deep security analysis of TANGRAM was made against known cryptanalytic approaches, including differential, linear, impossible differential, integral, and related-key cryptanalysis, the analysis shows that TANGRAM has sufficient security redundancy. Due to its bit-slice style, TANGRAM offers great performance in both hardware and software implementations, which provides good flexibility for different application platforms.

Key words: block cipher; bit-slice; security cryptanalysis; software performance; hardware performance; protection against side-channel attack

1 引言

TANGRAM, 就是七巧板, 它是我们设计的一族新的分组密码算法. TANGRAM 的主要设计思想是采用比特切片方法来设计适合多个软硬件平台的系列分组密码, 以灵活地适用于多种不同的应用场景. 这就像七巧板一样, 一套七巧板可以拼出多种不同的图形, 这是 TANGRAM 分组密码得名的原因.

本文在第2节给出 TANGRAM 的算法描述; 第3节给出 TANGRAM 的设计原则; 第4节给出我们对 TANGRAM 的安全性评估和分析结果; 第5节给出 TANGRAM 在多个软硬件环境下的实现结果; 第6节总结 TANGRAM 的优点和创新点.

2 算法描述

TANGRAM 是一族迭代型分组密码算法, 用 TANGRAM b/k 表示分组长度为 b 比特、密钥长度为 k 比特的分组密码版本. TANGRAM 包括三个版本: TANGRAM 128/128, TANGRAM 128/256, TANGRAM 256/256. 为方便起见, 按照分组长度的不同, 我们将分组长度为 128 和 256 比特的 TANGRAM 分组密码分别记作 TANGRAM-128 和 TANGRAM-256. 也就是说, TANGRAM-128 包含 TANGRAM 128/128 和 TANGRAM 128/256 两个分组密码, TANGRAM-256 指 TANGRAM 256/256.

TANGRAM 的整体结构为 SP 网络. TANGRAM 128/128 总轮数为 44 轮, TANGRAM 128/256 总轮数为 50 轮, TANGRAM 256/256 总轮数为 82 轮. 在加密算法的最后再增加一个子密钥异或操作. 每一轮变换包含三个步骤: 轮子密钥加 AddRoundKey (ARK), 列替换 SubColumn (SC), 行移位 ShiftRow (SR). 以 TANGRAM 128/128 为例, 用 $\mathcal{R}_i$ 表示在轮子密钥 K_i 作用下的轮变换, m 、 c 分别表示明文和密文, TANGRAM 128/128 的算法结构如图 1 所示.

2.1 密码状态和密钥状态

2.1.1 TANGRAM-128 状态

将 128 比特的明文、或 128 比特的中间状态、或 128 比特的密文统称为密码状态. 令 $W = w_{127} || \cdots || w_1 || w_0$ 表示一个密码状态, 我们用一个 4×32 的矩形比特阵列表示 W , 如图 2(a) 所示, 将前 32 个比特 $w_{31} || \cdots || w_1 || w_0$ 放在第 0 行, 接下来的 32 个比特 $w_{63} || \cdots || w_{33} || w_{32}$ 放在第 1 行, 依次类推. 类似

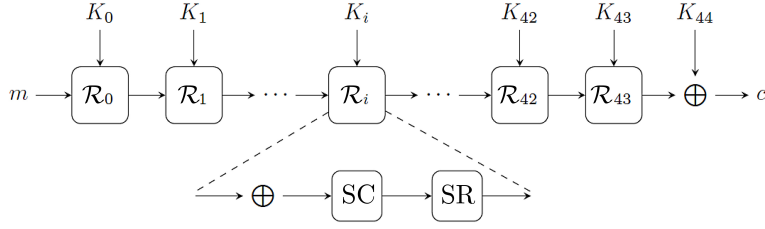


图 1 TANGRAM 128/128 的整体加密过程
Figure 1 Encryption process of TANGRAM 128/128

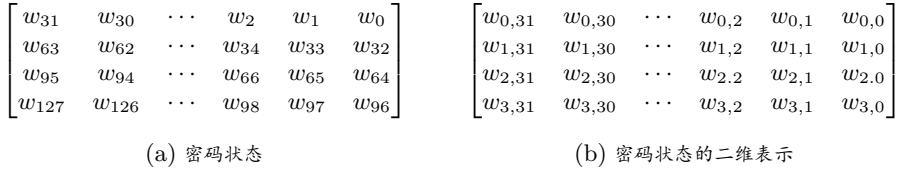


图 2 TANGRAM-128 的密码状态及其二维表示
Figure 2 Cipher state and its two-dimensional representation of TANGRAM-128

地, 128 比特的轮子密钥也用 一个 4×32 的矩形比特阵列来表示. 为了描述方便, 以下用二维形式表示 TANGRAM-128 的密码状态, 如图 2(b) 所示.

2.1.2 TANGRAM-256 状态

将 256 比特的明文、或 256 比特的中间状态、或 256 比特的密文统称为密码状态. 令 $W = w_{255} \parallel \cdots \parallel w_1 \parallel w_0$ 表示一个密码状态, 我们用一个 4×64 的矩形比特阵列表示 W , 如图 3(a) 所示, 将前 64 个比特 $w_{63} \parallel \cdots \parallel w_1 \parallel w_0$ 放在第 0 行, 接下来的 64 个比特 $w_{127} \parallel \cdots \parallel w_{65} \parallel w_{64}$ 放在第 1 行, 依次类推. 类似地, 256 比特的轮子密钥也用 一个 4×64 的矩形比特阵列来表示 W . 为了描述方便, 以下用二维形式表示 TANGRAM-256 的密码状态, 如图 3(b) 所示.

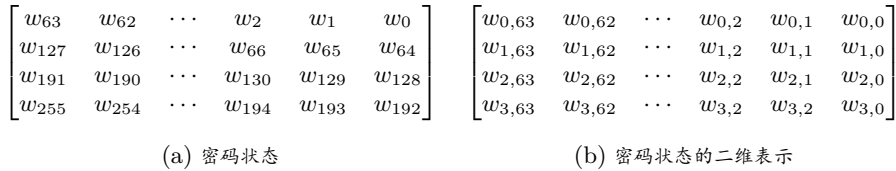


图 3 TANGRAM-256 的密码状态及其二维表示
Figure 3 Cipher state and its two-dimensional representation of TANGRAM-256

2.2 轮函数变换

2.2.1 轮密钥加 AddRoundKey

将 128 比特 (或 256 比特) 的轮子密钥逐比特异或到 128 比特 (或 256 比特) 的密码状态.

2.2.2 列替换 SubColumn

对每一列的 4 个比特做 S 盒替换. 以 TANGRAM-128 为例, 如图 4 所示, 令 $\text{Col}(j) = a_{3,j} \parallel a_{2,j} \parallel a_{1,j} \parallel a_{0,j} (0 \leq j \leq 31)$ 表示 S 盒的输入, $S(\text{Col}(j)) = b_{3,j} \parallel b_{2,j} \parallel b_{1,j} \parallel b_{0,j}$ 表示 S 盒的输出.

TANGRAM 的 S 盒是一个 4 比特到 4 比特的双射 $S: F_2^4 \rightarrow F_2^4$, S 的描述如表 1 所示 (十六进制).

2.2.3 行移位 ShiftRow

TANGRAM-128 与 TANGRAM-256 采用不同的移位参数, 分别用 ShiftRow_128 和 ShiftRow_256 表示.

TANGRAM-128 对每一行的 32 个比特做左循环移位. 第 0 行保持不动, 第 1 行左循环移动 1 位, 第 2 行左循环移动 8 位, 第 3 行左循环移动 11 位. 如图 5 所示, 其中 $\lll x$ 表示左循环移动 x 位.

表 1 TANGRAM 的 S 盒真值表
Table 1 Truth table of TANGRAM S-box

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S(x)$	8	0	3	E	7	D	C	2	6	F	5	9	A	1	B	4

$$\begin{pmatrix} a_{0,31} \\ a_{1,31} \\ a_{2,31} \\ a_{3,31} \end{pmatrix} \cdots \begin{pmatrix} a_{0,1} \\ a_{1,1} \\ a_{2,1} \\ a_{3,1} \end{pmatrix} \begin{pmatrix} a_{0,0} \\ a_{1,0} \\ a_{2,0} \\ a_{3,0} \end{pmatrix}$$

$$\downarrow S \quad \cdots \quad \downarrow S \quad \downarrow S$$

$$\begin{pmatrix} b_{0,31} \\ b_{1,31} \\ b_{2,31} \\ b_{3,31} \end{pmatrix} \cdots \begin{pmatrix} b_{0,1} \\ b_{1,1} \\ b_{2,1} \\ b_{3,1} \end{pmatrix} \begin{pmatrix} b_{0,0} \\ b_{1,0} \\ b_{2,0} \\ b_{3,0} \end{pmatrix}$$

图 4 TANGRAM-128 列替换操作

Figure 4 SubColumn operation of TANGRAM-128

$$\begin{aligned} (a_{0,31} \cdots a_{0,1} a_{0,0}) &\xrightarrow{\ll 0} (a_{0,31} \cdots a_{0,1} a_{0,0}) \\ (a_{1,31} \cdots a_{1,1} a_{1,0}) &\xrightarrow{\ll 1} (a_{1,30} \cdots a_{1,0} a_{1,31}) \\ (a_{2,31} \cdots a_{2,1} a_{2,0}) &\xrightarrow{\ll 8} (a_{2,23} \cdots a_{2,25} a_{2,24}) \\ (a_{3,31} \cdots a_{3,1} a_{3,0}) &\xrightarrow{\ll 11} (a_{3,20} \cdots a_{3,22} a_{3,21}) \end{aligned}$$

图 5 TANGRAM-128 行移位操作

Figure 5 ShiftRow operation of TANGRAM-128

TANGRAM-256 对每一行的 64 个比特做左循环移位. 第 0 行保持不动, 第 1 行左循环移动 1 位, 第 2 行左循环移动 8 位, 第 3 行左循环移动 41 位.

2.3 密钥扩展

2.3.1 TANGRAM 128/128 密钥扩展

将 128 比特的种子密钥 $V = v_{127} || \cdots || v_1 || v_0$ 用一个 4×32 的矩形比特阵列表示, 如图 6 所示.

$$\begin{bmatrix} v_{31} & \cdots & v_2 & v_1 & v_0 \\ v_{63} & \cdots & v_{34} & v_{33} & v_{32} \\ v_{95} & \cdots & v_{66} & v_{65} & v_{64} \\ v_{127} & \cdots & v_{98} & v_{97} & v_{96} \end{bmatrix} \quad \begin{bmatrix} k_{0,31} & \cdots & k_{0,2} & k_{0,1} & k_{0,0} \\ k_{1,31} & \cdots & k_{1,2} & k_{1,1} & k_{1,0} \\ k_{2,31} & \cdots & k_{2,2} & k_{2,1} & k_{2,0} \\ k_{3,31} & \cdots & k_{3,2} & k_{3,1} & k_{3,0} \end{bmatrix}$$

图 6 TANGRAM 128/128 密钥状态及其二维表示

Figure 6 Key state and its two-dimensional representation of TANGRAM 128/128

令 $\text{Row}_i = k_{i,31} || \cdots || k_{i,1} || k_{i,0}$ 表示第 i 行, $0 \leq i \leq 3$, Row_i 可以看作一个 32 比特的字. 在第 r ($r = 0, 1, \cdots, 43$) 轮, 先提取 128 比特的子密钥, 轮子密钥 $K_r = \text{Row}_3 || \text{Row}_2 || \text{Row}_1 || \text{Row}_0$; 然后, 128 比特的密钥状态做以下更新:

(1) 对密钥状态进行 S 盒操作 (与加密算法中的 S 盒相同), 即:

$$k'_{3,j} || k'_{2,j} || k'_{1,j} || k'_{0,j} := S(k_{3,j} || k_{2,j} || k_{1,j} || k_{0,j}), j = 0, 1, \cdots, 31$$

(2) 进行一轮的 4 分支广义 Feistel 变换, 如下:

$$\text{Row}'_0 := (\text{Row}_0 \lll 7) \oplus \text{Row}_1, \text{Row}'_1 := \text{Row}_2, \text{Row}'_2 := (\text{Row}_2 \lll 17) \oplus \text{Row}_3, \text{Row}'_3 := \text{Row}_0$$

(3) 将 6 比特的轮常数 $\text{RC}[r]$ ($r = 0, 1, \cdots, 43$) 与密钥状态第一行的 6 个比特 ($k_{0,5} || k_{0,4} || k_{0,3} || k_{0,2} || k_{0,1} || k_{0,0}$) 进行异或. 轮常数 $\text{RC}[r]$ ($r = 0, 1, \cdots, 43$) 的生成方法见 2.3.2 节.

将上面三个步骤的复合变换记为 $\text{Update}_{\text{RC}[r]}[X]$, 其中 X 为 128 比特. 在第 43 轮的密钥状态更新之后, 将更新的密钥状态的值赋给 K_{44} .

2.3.2 TANGRAM 128/256 密钥扩展

将 256 比特的种子密钥分成左右两半部分, $K = K_L || K_R$, 其中 K_L 和 K_R 都是 128 比特. 利用 Feistel 网络, TANGRAM 128/256 所需的 51 个 128 比特的子密钥生成如下:

$$K_0 = K_L, K_1 = K_R, K_{r+2} = K_r \oplus \text{Update}_{\text{RC}[r]}(K_{r+1}), r = 0, 1, \dots, 48$$

轮常数 $\text{RC}[r] (r = 0, 1, \dots, 48)$ 由一个 6 比特的线性反馈移位寄存器生成. 用 $(\text{rs}_5, \text{rs}_4, \text{rs}_3, \text{rs}_2, \text{rs}_1, \text{rs}_0)$ 表示反馈寄存器的状态, 在每一次更新时, 将状态左移动 1 位, 并将 rs_0 更新为 $\text{rs}_5 \oplus \text{rs}_4$, 初始值 $\text{RC}[0] := 0x1$.

2.3.3 TANGRAM 256/256 密钥扩展

将 256 比特的种子密钥 $V = v_{255} || \dots || v_1 || v_0$ 用一个 4×64 的矩形比特阵列表示, 如图 7 所示.

$$\begin{bmatrix} v_{63} & \cdots & v_2 & v_1 & v_0 \\ v_{127} & \cdots & v_{66} & v_{65} & v_{64} \\ v_{191} & \cdots & v_{130} & v_{129} & v_{128} \\ v_{255} & \cdots & v_{194} & v_{193} & v_{192} \end{bmatrix} \begin{bmatrix} k_{0,63} & \cdots & k_{0,2} & k_{0,1} & k_{0,0} \\ k_{1,63} & \cdots & k_{1,2} & k_{1,1} & k_{1,0} \\ k_{2,63} & \cdots & k_{2,2} & k_{2,1} & k_{2,0} \\ k_{3,63} & \cdots & k_{3,2} & k_{3,1} & k_{3,0} \end{bmatrix}$$

图 7 TANGRAM 256/256 密钥状态及其二维表示

Figure 7 Key state and its two-dimensional representation of TANGRAM 256/256

令 $\text{Row}_i = k_{i,63} || \dots || k_{i,1} || k_{i,0}$ 表示第 i 行, $0 \leq i \leq 3$, Row_i 可以看作一个 64 比特的字. 在第 $r (r = 0, 1, \dots, 81)$ 轮, 256 比特的轮子密钥 $K_r = \text{Row}_3 || \text{Row}_2 || \text{Row}_1 || \text{Row}_0$. 提取出轮子密钥 K_r 之后, 密钥状态做以下更新:

- (1) 对密钥状态进行 S 盒操作 (与加密算法中的 S 盒相同), 即:

$$k'_{3,j} || k'_{2,j} || k'_{1,j} || k'_{0,j} := S(k_{3,j} || k_{2,j} || k_{1,j} || k_{0,j}), j = 0, 1, \dots, 63$$

- (2) 进行一轮的 4 分支广义 Feistel 变换, 如下:

$$\text{Row}'_0 := (\text{Row}_0 \lll 7) \oplus \text{Row}_1, \text{Row}'_1 := \text{Row}_2, \text{Row}'_2 := (\text{Row}_2 \lll 17) \oplus \text{Row}_3, \text{Row}'_3 := \text{Row}_0$$

- (3) 对密钥状态的第一行的 7 个比特 $(k_{0,6} || k_{0,5} || k_{0,4} || k_{0,3} || k_{0,2} || k_{0,1} || k_{0,0})$, 异或 7 比特的轮常数 $\text{RC}[r] (r = 0, 1, \dots, 81)$.

最后, 将更新的密钥状态的值赋给 K_{82} .

轮常数 $\text{RC}[r] (r = 0, 1, \dots, 81)$ 由一个 7 比特的线性反馈移位寄存器生成. 用 $(\text{rs}_6, \text{rs}_5, \text{rs}_4, \text{rs}_3, \text{rs}_2, \text{rs}_1, \text{rs}_0)$ 表示反馈寄存器的状态, 在每一次更新时, 将状态左移动 1 位, 并将 rs_0 更新为 $\text{rs}_6 \oplus \text{rs}_5$, 初始值为 $\text{RC}[0] := 0x1$.

3 设计原理

3.1 TANGRAM 的整体设计原则

安全性: 密码算法的第一要素是安全性. 对于一个分组长度为 n 、密钥长度为 k 的分组密码, 如果不存在一种攻击方法有比 2^n 少得多的数据复杂度并且比 2^k 少得多的时间复杂度, 就称此分组密码是安全的. 通常用一个分组密码的安全冗余来评价其安全强度. 设分组密码的总轮数为 R , 如果最多可以攻击到 $R - m$ 轮的缩减轮密码, 就称该分组密码具有 m 轮的绝对安全冗余, 或者具有 m/R 的相对安全冗余. 安全冗余反映了一个分组密码抵抗现有的安全性分析方法的安全强度.

实现性能: 密码算法的第二要素是实现性能, 即在特定平台上进行加密运算或者解密运算所需要的资源. 在硬件实现中, 通常用实现面积和吞吐量来衡量; 在软件实现中, 通常用加密速度、代码量和内存占用

来衡量. 处理器字长和指令集的差异, 会让一个密码算法的性能在一些平台上表现优秀、而在另外一些平台上表现很差. 考虑到一个分组密码具有多种多样的应用场合, 我们在设计 TANGRAM 时, 兼顾硬件和软件实现, 尽可能地使 TANGRAM 在多个平台上都具有不错表现.

简单性: 一个简单的密码算法, 由于其描述和运算的简单性, 会利于正确无误的实现, 也会吸引更多人对安全性和实现性能的关注, 进而对该密码算法建立更强的信任感. 因此, 简单性是我们设计 TANGRAM 的一项原则.

3.2 TANGRAM 轮函数的设计

TANGRAM 的设计基于比特切片方法^[1], 整体结构为 SP 网络. 考虑一个分组长度为 128 比特或 256 比特的 SP 型分组密码, 设 S 层由 32 个或 64 个 4×4 的 S 盒并置构成. 因而在比特切片的实现中, 子块的长度为 32 比特或 64 比特. 将 128 比特或 256 比特的密码状态用一个 4×32 或 4×64 的比特阵列表示, S 层对每一列独立地进行相同的 S 盒替换, 那么, 接下来的 P 变换应该让输出的每一列依赖于其它一些列, 以提供必要的扩散性. 我们选择了对每一行的 32 比特字或 64 比特字进行循环移位操作: 循环移位位在硬件上可以用简单的拉线实现; 能够达到每一列依赖于其它一些列的目的; 比特切片实现时, 循环移位位在软件上可以很容易的高效实现. 至此, 我们得到了 TANGRAM 轮函数的基本框架.

得益于我们对 S 盒的细致选取 (详见 3.3 节), TANGRAM 达到了很好的安全性和实现性能的平衡; P 置换的设计也非常重要 (详见 3.4 节), 它仅由三个 32 比特字或 64 比特字上的循环移位组成, 不仅能提供必要的扩散性, 也兼顾了硬件和软件的实现性能. TANGRAM 的 S 盒层和 P 置换层组合在一起, 所形成的密码算法整体具有很好的抵抗差分/线性攻击, 这也在很大程度上提高了 TANGRAM 的安全性和实现性能之间的性价比.

3.3 TANGRAM 的 S 盒设计

3.3.1 相关定义和定理

定义 1 令 S 表示一个 4×4 的 S 盒, 对任意 $\Delta I, \Delta O \in F_2^4$, 定义 $ND_S(\Delta I, \Delta O)$ 为:

$$ND_S(\Delta I, \Delta O) := \#\{x \in F_2^4 | S(x) \oplus S(x \oplus \Delta I) = \Delta O\}$$

定义 2 (差分均匀度) 令 S 表示一个 4×4 的 S 盒, 定义 S 的差分均匀度 $\text{Diff}(S)$ 为:

$$\text{Diff}(S) := \max_{\Delta I \neq 0, \Delta O} = ND_S(\Delta I, \Delta O)$$

定义 3 令 S 表示一个 4×4 的 S 盒, 对任意 $\Gamma I, \Gamma O \in F_2^4$, 定义 $\text{Imb}_s(\Gamma I, \Gamma O)$ 为:

$$\text{Imb}_s(\Gamma I, \Gamma O) := |\#\{x \in F_2^4 | \Gamma I \cdot x = \Gamma O \cdot S(x)\} - 8|$$

其中 “ $\cdot$ ” 表示 F_2^4 上的内积.

定义 4 (线性度) 令 S 表示一个 4×4 的 S 盒, 定义 S 的线性度 $\text{Lin}(S)$ 为:

$$\text{Lin}(S) := \max_{\Gamma I, \Gamma O \neq 0} 2 \text{Imb}_s(\Gamma I, \Gamma O)$$

对于任意 4×4 的双射 S 盒, 均有 $\text{Diff}(S) \geq 4$ 并且 $\text{Lin}(S) \geq 8$, 使得这两个值都达到最小值的 S 盒被称为最优 S 盒.

定义 5 (最优 S 盒^[2]) 令 S 表示一个 4×4 的 S 盒, 称 S 是最优 S 盒, 若它满足以下三个条件:

- (1) S 是双射的;
- (2) $\text{Diff}(S) = 4$;
- (3) $\text{Lin}(S) = 8$.

定义 6 (SetD1_S 和 CarD1_S) 令 $\Delta I \in F_2^4$ 表示一个非零输入差分, $\Delta O \in F_2^4$ 表示一个非零输出差分. 令 $\text{wt}(x)$ 表示 x 的比特汉明重量. 定义 SetD1_S 为:

$$\text{SetD1}_S := \{(\Delta I, \Delta O) \in F_2^4 \times F_2^4 \mid \text{wt}(\Delta I) = \text{wt}(\Delta O) = 1 \text{ and } \text{ND}_S(\Delta I, \Delta O) \neq 0\}$$

令 CarD1_S 表示集合 SetD1_S 的势.

定义 7 (SetL1_S 和 CarL1_S) 令 $\Gamma I \in F_2^4$ 表示一个非零输入选取模式 (也称为输入掩码), $\Gamma O \in F_2^4$ 表示一个非零输出选取模式 (也称为输出掩码). 定义 SetL1_S 为:

$$\text{SetL1}_S := \{(\Gamma I, \Gamma O) \in F_2^4 \times F_2^4 \mid \text{wt}(\Gamma I) = \text{wt}(\Gamma O) = 1 \text{ and } \text{Imb}_s(\Gamma I, \Gamma O) \neq 0\}$$

令 CarL1_S 表示集合 SetL1_S 的势.

定义 8 (仿射等价 [2]) 两个 4×4 的 S 盒 S 和 S' 被称为仿射等价, 如果存在双射的线性映射 A, B 以及常数 $a, b \in F_2^4$, 满足

$$S'(x) = B(S(A(x) + a)) + b$$

若两个 S 盒 S 和 S' 仿射等价, 则有 $\text{Diff}(S) = \text{Diff}(S')$ 和 $\text{Lin}(S) = \text{Lin}(S')$. 也就是说, S 盒的差分均匀度和线性度在仿射变换下是不变量.

定理 1 [2] 令 S 和 S' 是两个仿射等价的 S 盒. 若 S 是最优 S 盒, 则 S' 也是最优 S' 盒.

定义 9 (置换异或等价 permutation-then-XOR equivalent [2]) 两个 4×4 的 S 盒 S 和 S' 被称为置换异或等价, 如果存在 F_2 上的 4×4 置换矩阵 P_0, P_1 以及常量 $a, b \in F_2^4$, 满足

$$S'(x) = P_1(S(P_0(x) + a)) + b$$

我们简称这种等价为 PE 等价 (PE equivalence). 若两个 S 盒 PE 等价, 则它们一定仿射等价.

定义 10 (分量布尔函数) 令 S 表示一个 4×4 的 S 盒. 对于任意的 $b \in F_2^4$, 如下定义 S 的分量布尔函数 S_b :

$$S_b : F_2^4 \rightarrow F_2 \quad (1)$$

$$S_b(x) := x \mapsto b \cdot S(x) \quad (2)$$

3.3.2 S 盒的选取

TANGRAM 采用了 4 比特 S 盒, 设计准则如下:

- (1) S 是双射的, 即对任意 $x \neq x'$, 有 $S(x) \neq S(x')$;
- (2) 对任意非零输入差分 $\Delta I \in F_2^4$ 和非零输出差分 $\Delta O \in F_2^4$, $\text{ND}_S(\Delta I, \Delta O) \leq 4$;
- (3) $\text{CarD1}_S = 2$;
- (4) 对任意非零输入选取模式 $\Gamma I \in F_2^4$ 和非零输出选取模式 $\Gamma O \in F_2^4$, $\text{Imb}_s(\Gamma I, \Gamma O) \leq 4$;
- (5) $\text{CarL1}_S = 2$;
- (6) S 没有不动点, 即对任意 $x \in F_2^4$, 有 $S(x) \neq x$.

一个 S 盒若满足准则 (1)、(2) 和 (4), 那么它是最优的 S 盒, 并且任意一个与之仿射等价的 S 盒也满足准则 (1)、(2) 和 (4)(见定义 8 和定理 1). 一个 S 盒若满足准则 (1) ~ (5), 那么, 任意一个与之 PE 等价 (见定义 9) 的 S 盒也满足准则 (1) ~ (5).

根据定理 1, 所有的最优 S 盒可以通过仿射等价关系划分等价类. 实际上, 所有的最优 4 比特 S 盒只存在 16 种不同的仿射等价类 [2]. 利用文献 [2] 中给出的 16 个仿射等价类的代表元, 我们设计了一个算法将所有满足准则 (1) ~ (5) 的 4 比特 S 盒划分成 PE 等价类, 结果显示满足条件的只有 4 个 PE 类.

表2给出了这4个PE类的代表元,其中,每一行中,第一个数代表输入为0的S盒的输出,第二个数代表输入为1的S盒的输出,依次类推.

表2 满足准则(1)~(5)的4个PE类的代表元
Table 2 Representatives for all 4 PE classes fulfilling criteria (1)~(5)

PE_0	6, 0, 8, 15, 12, 3, 7, 13, 11, 14, 1, 4, 5, 9, 10, 2
PE_1	3, 2, 8, 13, 15, 5, 6, 10, 9, 14, 4, 7, 0, 12, 11, 1
PE_2	6, 8, 15, 4, 12, 7, 9, 3, 11, 1, 0, 14, 5, 10, 2, 13
PE_3	8, 1, 6, 12, 5, 15, 10, 3, 7, 11, 13, 2, 0, 14, 9, 4

如果一个S盒满足准则(1)~(5),那么给它的输入变量或者输出变量异或一个常数,所得到的新S盒仍然满足准则(1)~(5),并且也不会改变相应密码算法的最优差分迹的概率和最优线性迹的相关系数(差分迹及其概率、线性迹及其相关系数,详见第4节的安全性分析).因此,我们只需考虑表1中的4个代表元所生成的 $4 \times 4! \times 4! = 2304$ 个S盒,用 $\{S_i : i = 0, 1, \dots, 2303\}$ 来表示这2304个S盒.

首先,利用文献[3]中的算法2,能够删除掉一部分会导致每轮只有一个差分(或者线性)活跃S盒的候选S盒.总共有1776个S盒被删除,只剩下528个候选S盒.接下来,我们通过考查相应密码算法抵抗差分和线性密码分析的安全性来对这528个S盒做进一步筛选.令 $Prob_{D25}$ 表示最优25轮差分迹的概率, Cor_{L25}^2 表示最优25轮线性迹的相关势(相关势的定义见第4节的安全性分析).固定P置换ShiftRow_128,对于528个候选S盒中的每一个,检验相应的128比特分组长度下的密码算法是否满足以下两个不等式: $Prob_{D25} < 2^{-128}$ 和 $Cor_{L25}^2 < 2^{-128}$.实验结果表明,528个S盒中,有124个S盒满足前者不等式;而在这124个S盒中,又有36个满足后者不等式.根据 $Prob_{D25}$ 和 Cor_{L25}^2 的值,这36个S盒被划分成2组,表3给出了 $Prob_{D25}$ 和 Cor_{L25}^2 的值以及对应S盒的个数.

表3 32个候选S盒分成2个组
Table 3 Divide 32 candidate S-boxes into two groups

Group	$Prob_{D25}$	Cor_{L25}^2	Number of S-boxes
1	130	136	28
2	130	134	8

在这两组S盒中,由于第1组的最优25轮线性迹相关势更高,我们选择了第1组的28个S盒进行256比特版本密码算法的测试.固定P置换ShiftRow_256,对于28个候选S盒中的每一个,我们搜索其在相应的256比特分组长度下的密码算法的最优差分迹与最优线性迹.结果表明49轮的最优差分迹概率和最优线性迹相关势均小于 2^{-256} .

我们选取了第1组S盒中的第1个S盒.通过在这个S盒变换的前后各加一个常数,可以得到 $16 \times 16 = 256$ 个PE等价的S盒.在这256个S盒中,我们选择了一个没有不动点、并且软件实现速度和硬件面积需求均有较好表现的S盒作为TANGRAM的S盒.

3.4 TANGRAM的扩散层设计

扩散层的设计旨在提供扩散性,使输出的每一列依赖于输入的一些列.我们所选的设计是对每一行的32个比特或64个比特做左循环移位,令 c_i ($i = 0, 1, 2, 3$)表示对第 i 行左循环移位的参数,这些参数的选取准则如下:

- (1) $c_0 = 1, c_1 = 1, c_2 = 8, c_2 < c_3$;
- (2) 达到全扩散所需的轮数最少.

注:考虑到在一些低端处理器上,循环左移的参数选为1和8时更为高效,因此我们选取了 $c_1 = 1$ 且 $c_2 = 8$.

对于TANGRAM-128,我们的实验结果表明,共有5组候选参数满足以上准则,经过6轮的轮函数变换之后,128个输入比特中的每个比特都会影响到128个输出比特中的每个比特.从这5种候选参数中,我们选取了 $(c_1, c_2, c_3) = (1, 8, 11)$ 作为ShiftRow的循环移位参数.

对于 TANGRAM-256, 我们的实验结果表明, 共有 13 组候选参数满足以上准则, 经过 8 轮的轮函数变换之后, 256 个输入比特中的每个比特都会影响到 256 个输出比特中的每个比特. 从这 13 种候选参数中, 我们选取了 $(c_1, c_2, c_3) = (1, 8, 41)$ 作为 ShiftRow 的循环移位参数.

3.5 TANGRAM 的密钥扩展算法设计

TANGRAM 的密钥扩展算法设计准则如下:

- (1) 共用加密算法中的 S 盒, 减少实现代价;
- (2) 扩散层使用了广义 Feistel 结构, 增加其扩散性;
- (3) 使用轮常数消除对称性.

TANGRAM 的密钥扩展算法满足以上设计准则. 通过一些初步分析, 我们相信 TANGRAM 能够抵抗与密钥扩展算法有关的攻击

4 安全性分析

4.1 TANGRAM 安全性概述和声明

TANGRAM 128/128 的总轮数为 44 轮, 根据我们的安全性分析和评估结果, 我们最多能攻击到 28 轮 (占全部轮数的 63.6%). 因此, TANGRAM 128/128 的绝对安全冗余为 16 轮, 相对安全冗余为 36.4%.

6 轮 TANGRAM-128 达到全扩散, 因此我们在 44 轮的基础上又增加了 6 轮, 将 50 轮作为 TANGRAM 128/256 的总轮数. 根据我们对 28 轮 TANGRAM 128/128 的差分攻击结果, 我们估计, 对 TANGRAM 128/256 最多能攻击到 30 轮 (占全部轮数的 60%). 因此, TANGRAM 128/256 的绝对安全冗余为 20 轮, 相对安全冗余为 40%.

TANGRAM 256/256 的总轮数为 82 轮, 我们估计最多能攻击到 56 轮 (占全部轮数的 68%). 因此, TANGRAM 256/256 的绝对安全冗余为 26 轮, 相对安全冗余为 32%.

我们认为: 以当前分组密码分析方法的发展现状作为参考, 我们为 TANGRAM 的 3 个版本预留的冗余都足够保证其抵抗数学类攻击的安全性.

4.2 TANGRAM 抵抗差分密码分析的安全性

利用差分密码分析方法^[4,5]攻击一个分组长度为 n 比特的 R 轮分组密码, 需要找到一个 $R - w$ 轮的、并且概率大于 2^{1-n} 的差分传播, w 的取值和具体分组密码有关. 差分迹也称为差分特征. 一个差分传播是若干差分迹的集合, 集合中的差分迹具有相同的输入差分 and 相同的 $R - w$ 轮变换后的输出差分, 差分传播的概率是集合中所有差分迹的概率之和.

基于我们提出的搜索算法^[6], 我们搜索了 128 比特分组长度下 TANGRAM 的第 1–26 轮的最优差分迹和 256 比特分组长度下 TANGRAM 的第 1–50 轮的最优差分迹, 结果如表 4 和表 5 所示.

表 4 TANGRAM-128 最优差分迹概率
Table 4 Probabilities of the best differential trails of TANGRAM-128

轮数	Prob.	轮数	Prob.	轮数	Prob.	轮数	Prob.	轮数	Prob.
1	2	7	25	13	66	19	98	25	130
2	4	8	32	14	71	20	103	26	135
3	7	9	40	15	76	21	108		
4	10	10	49	16	82	22	114		
5	14	11	55	17	87	23	119		
6	18	12	60	18	92	24	124		

从表 4 可以看到, 128 比特分组长度下 TANGRAM 的 25 轮最优差分迹的概率为 2^{-130} , 小于边界值 2^{-127} ; 从表 5 可以看到, 256 比特分组长度下 TANGRAM 的 49 轮最优差分迹的概率为 2^{-258} , 小于边界值 2^{-255} .

TANGRAM 扩散层的设计非常简单, 因此需要考虑 TANGRAM 差分/线性迹的聚集效应. TANGRAM 和 RECTANGLE 在设计上具有相似性. 我们对 RECTANGLE 差分迹的聚集效应进行了深入

表 5 TANGRAM-256 最优差分迹概率
Table 5 Probabilities of the best differential trails of TANGRAM-256

轮数	Prob.	轮数	Prob.	轮数	Prob.	轮数	Prob.	轮数	Prob.
1	2	11	55	21	108	31	162	41	215
2	4	12	60	22	114	32	167	42	220
3	7	13	66	23	119	33	172	43	226
4	10	14	71	24	124	34	178	44	231
5	14	15	76	25	130	35	183	45	236
6	18	16	82	26	135	36	188	46	242
7	25	17	87	27	140	37	194	47	247
8	32	18	92	28	146	38	199	48	252
9	40	19	98	29	151	39	204	49	258
10	49	20	103	30	156	40	210	50	263

研究和实验验证, 结果表明: RECTANGLE 差分迹的聚集效应非常有限, 即使利用聚集效应, 也不能构造出轮数更长的差分区分器. 可以推断: TANGRAM 同样具有非常有限的差分迹聚集效应. RECTANGLE 的分组长度为 64 比特, 而 TANGRAM 的分组长度为 128 或 256 比特, 因此受限于计算机的处理能力, 对 TANGRAM 差分/线性迹聚集的实验在实际中几乎不可行. 我们为 TANGRAM 预留了足够的安全冗余, 因此我们相信三个版本的 TANGRAM 分组密码均足以抵抗差分密码分析.

4.3 对缩减轮 TANGRAM 的差分攻击

利用 TANGRAM-128 的一个概率为 2^{-124} 的 24 轮差分迹, 我们给出了一个对 28 轮 TANGRAM 128/128 的差分攻击. 攻击的数据复杂度为 $2^{126.68}$ 个明文对, 存储复杂度为 2^{120} 个密钥计数器, 时间复杂度大约为 $2^{122.5}$ 次 28 轮加密, 成功率大约为 82%.

对 TANGRAM128/128, 在我们所考虑的安全性分析方法中, 上面对 28 轮的差分攻击, 是能攻击到的最长轮数. 根据这个攻击, 我们估计, 对 TANGRAM128/256, 最多能攻击到 30 轮.

4.4 TANGRAM 抵抗线性密码分析的安全性

令一个线性迹 (也称为线性逼近, 或者线性特征) 成立的概率为 p , 定义它的偏差 ε 为 $(p - 1/2)$, 相关系数 C 为 2ε . 利用线性密码分析方法^[7,8] 攻击一个分组长度为 n 比特的 R 轮分组密码, 需要找到一个 $R - u$ 轮的、并且相关系数的绝对值大于 $2^{-n/2}$ 的线性传播, u 的取值和具体分组密码有关. 一个线性传播是若干线性迹的集合, 集合中的线性迹具有相同的输入选取模式和相同的 $R - u$ 轮变换后的输出选取模式, 线性传播的相关系数是集合中所有线性迹的相关系数之和. 线性迹的相关系数带有正负号, 是正号还是负号, 这和子密钥的取值有关.

表 6 TANGRAM-128 最优线性迹的相关势
Table 6 Correlation potentials of the best linear trails of TANGRAM-128

轮数	Cor.Pot.	轮数	Cor.Pot.	轮数	Cor.Pot.	轮数	Cor.Pot.	轮数	Cor.Pot.
1	2	6	20	11	52	16	82	21	112
2	4	7	26	12	58	17	88	22	118
3	8	8	34	13	64	18	94	23	124
4	12	9	40	14	70	19	100	24	130
5	16	10	46	15	76	20	106	25	136

修改最优差分迹的搜索算法, 我们搜索了 128 比特分组长度下 TANGRAM 的第 1-25 轮的最优线性迹和 256 比特分组长度下 TANGRAM 的第 1-29 轮的最优线性迹, 结果如表 6 和表 7 所示.

从表 6 可以看到, 24 轮的最优线性迹的相关势为 2^{-130} , 小于边界值 2^{-128} ; 由于计算能力所限, 256 比特分组长度的最优线性迹只跑到 29 轮, 但根据表 7 所呈现的规律: 从第 8 轮开始, 每增加一轮, 则线

表 7 TANGRAM-256 最优线性迹的相关势
Table 7 Correlation potentials of the best linear trails of TANGRAM-256

轮数	Cor.Pot.	轮数	Cor.Pot.	轮数	Cor.Pot.	轮数	Cor.Pot.	轮数	Cor.Pot.
1	2	7	26	13	64	19	100	25	136
2	4	8	34	14	70	20	106	26	142
3	8	9	40	15	76	21	112	27	148
4	12	10	46	16	82	22	118	28	154
5	16	11	52	17	88	23	124	29	160
6	20	12	58	18	94	24	130		

性相关势的重量增加了 6, 因此, 可以推断 46 轮的最优线性迹的相关势为 2^{-262} , 小于边界值 2^{-256} .

我们对 RECTANGLE 线形迹的聚集效应进行了深入研究和实验验证, 结果表明: RECTANGLE 线形迹的聚集效应也非常有限. 同样, 可以推断: TANGRAM 也具有非常有限的线性迹聚集效应. 考虑到我们为 TANGRAM 预留的安全冗余, 我们相信三个版本的 TANGRAM 分组密码均足以抵抗线性密码分析.

4.5 TANGRAM 抵抗不可能差分密码分析的安全性

不可能差分密码分析^[9]利用概率为 0 的差分传播, 这种差分传播一般通过中间相遇方法构造. 根据不可能差分密码分析的研究现状, 一个分组密码的最长不可能差分区分器的轮数接近于全扩散轮数的两倍.

比如, RECTANGLE 的全扩散轮数是 4 轮, 我们能找到的它的最长不可能差分区分器的轮数是 8 轮.

由于 TANGRAM-128 和 TANGRAM-256 的全扩散轮数分别是 6 轮和 8 轮, 因此我们预估它们的最长不可能差分区分器的轮数大约为 12 轮和 16 轮, 这与相应 TANGRAM 版本的总轮数相差很大. 因此 TANGRAM 的三个版本均具有足够的抵抗不可能差分密码分析的安全性.

4.6 TANGRAM 抵抗积分密码分析的安全性

积分密码分析 (也称为平方攻击, 饱和攻击)^[10,11]关注的是多个数据之和的传播性质, 积分区分器的概率也为 1. 可分性^[12]是近几年提出的一种一般化积分性质, 利用可分性可以构造较传统方法更长更精确的积分区分器. 我们将采取基于混合整数线性规划的技术^[13]搜索 TANGRAM 系列算法的积分区分器.

搜索结果显示: TANGRAM-128 存在 12 轮积分区分器, 并且在混合整数线性规划的搜索框架下不存在 13 轮积分区分器; TANGRAM-256 存在 16 轮积分区分器, 但是受限于计算资源, 我们无法确定 TANGRAM-256 是否存在 17 轮积分区分器. 虽然搜索算法无法返回 TANGRAM-256 的 17 轮结果, 但是基于可分性在其它分组密码上的应用, 我们相信 TANGRAM-256 的积分区分器应该不超过 20 轮.

TANGRAM-128 的 12 轮积分区分器: 固定 TANGRAM-128 最左边 S 盒的 4 比特为任意常数, 遍历剩下的 124 比特得到一组包含 2^{124} 个数据的明文集, 将该集合加密 12 轮后所有输出的异或在 $w_{3,20}$ 和 $w_{3,9}$ 处取值恒为零. TANGRAM-256 的 16 轮积分区分器: 固定 TANGRAM-256 最左边 S 盒的高位比特 (即 $w_{3,61}$) 为任意常数, 遍历剩下 255 比特得到一组包含 2^{255} 个数据的明文集, 将该集合加密 16 轮后所有输出的异或在 $w_{3,61}$ 处取值恒为零.

同样, 我们构造出的可分积分区分器的轮数和相应 TANGRAM 版本的总轮数相差很大. 因此 TANGRAM 的三个版本均具有足够的抵抗积分密码分析的安全性.

4.7 抗相关密钥密码分析

在利用密钥扩展算法的攻击中, 最有效的是滑动攻击^[14,15]和相关密钥攻击^[16]. 对于 TANGRAM, 在密钥扩展算法中嵌入不同的轮常量可以阻止滑动攻击; 类似于加密算法中的 S 盒层的使用, 给密钥扩展算法提供足够的混淆性; 一轮广义 Feistel 网络可以提供适合的扩散性. 基于上述考虑, 我们相信, TANGRAM 不存在弱密钥、等价密钥, 对密钥的选择没有限制, 其三个版本均具有足够的抵抗密钥扩展类攻击的安全性.

4.8 抗代数攻击

代数攻击应用于分组密码的成功例子非常罕见. TANGRAM 的 S 盒没有特殊的代数结构. 因此, 我们相信代数攻击对 TANGRAM 的安全性没有任何威胁.

5 软硬件性能分析

5.1 在 X64 平台的软件实现

5.1.1 测试环境说明

测试的硬件环境是一台个人计算机, 具有 3.40 GHz 的 Intel(R) Core(TM) i7-6700 CPU, 16.0 GB 的内存. 软件环境是 64 位 Windows 10 操作系统, 使用 Microsoft Visual Studio 2010 Professional Edition, Intel C/C++ 编译器. 在所有测试数据中, 测试消息的长度为 2 KB, cpb 表示 cycles per byte, Mbps 表示 Mbits per second.

5.1.2 测试结果

对 TANGRAM 的三个版本, 我们进行了 ECB、CBC 和 CTR 三种工作模式下的基本实现和 SSE/AVX 指令集的优化实现.

表 8 ECB 模式速度测试结果
Table 8 Software implementation speed in ECB mode

算法软件实现类别		加密速率		加密速率	
		cpb	Mbps	cpb	Mbps
64 bit Windows 环境下的算法速率 优化 C 实现	基本实现	128/128	23.4	1191.8	27.8
		128/256	26.4	1055.0	32.0
		256/256	22.0	1267.2	27.0
	SSE 指令集 优化实现	128/128	7.3	3815.5	8.1
		128/256	8.3	3355.8	9.4
		256/256	15.3	1820.4	17.2
	AVX 指令集 优化实现	128/128	4.7	5926.1	5.1
		128/256	5.3	5255.2	5.8
		256/256	9.1	3060.7	9.4

表 9 CBC 模式速度测试结果
Table 9 Software implementation speed in CBC mode

算法软件实现类别		加密速率		加密速率	
		cpb	Mbps	cpb	Mbps
64 bit Windows 环境下的算法速率 优化 C 实现	基本实现	128/128	24.4	1142.2	28.1
		128/256	27.8	1002.9	32.4
		256/256	22.0	1267.2	27.0

利用 SSE 指令集, 对于 TANGRAM-128, 每次完成 4 个分组块的并行运算; 对于 TANGRAM-256, 每次完成 2 个分组块的并行运算. 利用 AVX 指令集, 对于 TANGRAM-128, 每次完成 8 个分组块的并行运算; 对于 TANGRAM-256, 每次完成 4 个分组块的并行运算. 由于 TANGRAM 的设计基于比特切片技术, 以上数据格式转换的开销非常低 (使用 (v)punpck 系列指令实现).

表 8-表 10 列出了我们的测试结果. 可以看出, 利用 SSE 和 AVX 指令集, TANGRAM 的软件实现速度得到了数倍的提升.

表 10 CTR 模式速度测试结果
Table 10 Software implementation speed in CTR mode

算法软件实现类别			加密速率		解密速率	
			cpb	Mbps	cpb	Mbps
64 bit Windows 环境下的算法速率 优化 C 实现	SSE 指令集 优化实现	128/128	7.2	3868.4	7.2	3868.4
		128/256	8.4	3315.8	8.3	3355.8
		256/256	16.7	1667.8	16.7	1667.8
	AVX 指令集 优化实现	128/128	4.7	5926.1	4.6	6055.0
		128/256	5.2	5356.3	5.2	5356.3
		256/256	8.9	3129.5	8.9	3129.5

5.2 在 32 位 ARM Cortex-M3 平台的软件实现

在 32 位 ARM Cortex-M3 平台的实现, 采用 Keil5 + Jlink 进行编译调试, 性能测试采用 Keil5 软件记录时间, 通过比较执行代码片段前后的时间差得到运行时间. ARM 平台拥有丰富的指令集, 使用如多内存加载、循环移位、逻辑运算、字节交换等指令可以非常方便地实现 TANGRAM 算法. 具体测试结果如表 11 所示, 我们给出了两种不同速度单位分别在 C 语言和汇编语言下的测试结果, cpb (cycles per byte) 由设备频率 72 MHz 换算而来.

表 11 32 位 ARM 环境下的算法速率优化 C 实现速度测试结果
Table 11 Speed-optimized C implementation results on 32-bit ARM

速率	TANGRAM 128/128		TANGRAM 128/256		TANGRAM 256/256	
	cpb	Mbps	cpb	Mbps	cpb	Mbps
加密速率 (C)	270.4	2.13	325.4	1.77	570.3	1.01
解密速率 (C)	259.5	2.22	314.8	1.83	581.8	0.99
加密速率 (ASM)	145.1	3.97	184.0	3.13	439.7	1.31
解密速率 (ASM)	142.9	4.03	181.7	3.17	426.7	1.35

此外, NEON 是适用于 ARM Cortex-A 系列处理器的一种 128 位 SIMD (single instruction, multiple data, 单指令、多数据) 扩展结构. 类似于 Intel 处理器中的 SSE 指令, 利用 NEON 指令集可以更大幅度地提升 TANGRAM 在 ARM 平台的加解密速度.

5.3 在 8 位 AVR 平台的软件实现

为了评估 TANGRAM 在 8 位微控制器上的性能, 我们进行了针对 Atmel AVR 8 位微处理器的软件实现. 具体的目标设备是 ATmega128, 其上配有 128 KBytes 的闪存 (flash, 即只读内存 ROM), 4 KBytes 的随机存储内存 (RAM), 以及 32 个 8 比特通用寄存器. 实现所考虑的应用场景包括:

- (1) 使用 CBC 模式加/解密 128 字节消息. 这种应用场景对应于传感器网络和 IoT 设备之间的安全通信.
- (2) 使用 CTR 模式加密一个分组块的消息 (128 比特或 256 比特). 这种应用场景对应于 IoT 认证所需执行的挑战-握手认证协议.

实现所使用的是汇编代码, 并使用 Atmel Studio 7.0 中所带有的 AVR macro assembler 进行编译. 我们关注以下三个方面: ROM 开销 (以字节 Bytes 记), RAM 开销 (以字节 Bytes 记), 执行时间 (以 cycles 记), 以及执行速度 (以 cycles/byte 记). 表 12~表 15 汇总了我们的测试结果.

由测试结果可以看出, 对于 128 比特分组长度的 TANGRAM, 当考虑第一种应用场景 (CBC 模式加密解密 128 字节数据), 其可以在不损失运行时间的情况下花费极小的代码量 (ROM); 对于所考虑的第二

表 12 TANGRAM 在 8 位 AVR 平台的测试结果 (CBC 模式下的加密算法)
Table 12 Results on 8-bit AVR platform (encryption speed in CBC mode)

CBC 模式	消息长度 [B.]	主函数框架		加密算法			
		ROM [B.]	RAM [B.]	ROM [B.]	RAM [B.]	执行时间 [cycle]	速度 [cpb]
TANGRAM 128/128	128	72	160	408	720	53 331	416.65
TANGRAM 128/256	128	72	176	408	816	60 435	472.15
TANGRAM 256/256	128	72	192	436	2672	144 076	1125.59

表 13 TANGRAM 在 8 位 AVR 平台的测试结果 (CBC 模式下的解密算法)
Table 13 Results on 8-bit AVR platform (decryption speed in CBC mode)

CBC 模式	消息长度 [B.]	主函数框架		解密算法			
		ROM [B.]	RAM [B.]	ROM [B.]	RAM [B.]	执行时间 [cycle]	速度 [cpb]
TANGRAM 128/128	128	72	160	458	720	55 308	432.09
TANGRAM 128/256	128	72	176	458	816	62 652	489.47
TANGRAM 256/256	128	72	192	930	2672	148 930	1163.52

表 14 TANGRAM 在 8 位 AVR 平台的测试结果 (CBC 模式下的密钥扩展 + 加密 + 解密)
Table 14 Results on 8-bit AVR platform (speed of combination of key-schedule+encryption+decryption in CBC mode)

CBC 模式	消息长度 [B.]	主函数框架		密钥扩展 + 加密 + 解密		
		ROM [B.]	RAM [B.]	ROM [B.]	RAM [B.]	执行时间 [cycle]
TANGRAM 128/128	128	72	160	946	720	114 851
TANGRAM 128/256	128	72	176	1058	816	132 087
TANGRAM 256/256	128	72	192	1722	2672	325 896

表 15 TANGRAM 在 8 位 AVR 平台的测试结果 (CTR 模式)
Table 15 Results on 8-bit AVR platform (in CTR mode)

CTR 模式	消息长度 [B.]	主函数框架		加密算法			
		ROM [B.]	RAM [B.]	ROM [B.]	RAM [B.]	执行时间 [cycle]	速度 [cpb]
TANGRAM 128/128	16	50	48	1174	0	6794	424.63
TANGRAM 128/256	16	50	64	1270	0	7694	480.88
TANGRAM 256/256	32	50	96	3470	16	45 211	1412.84

种应用场景 (CTR 模式执行认证握手协议处理较短数据), 其只需要可忽略的随机存储内存 (RAM). 实际上, 第一种应用场景中几乎全部的 RAM 是用来存储扩展出来的轮子密钥; 类似地, 在第二种应用场景下很大一部分的 ROM 需求是用来存储扩展出来的轮子密钥, 而加密解密部分所需的代码量和随机存储内存 (ROM 和 RAM) 都非常少. 对于 256 比特分组长度的 TANGRAM, 由于状态大到无法全部存放于寄存器上进行更新, 我们需要将一半的状态存储于 RAM 中, 这不仅需要额外的 RAM 开销, 而且需要不断地从 RAM 中装载和暂存一半的状态字节, 这导致执行时间也有了不容忽视的额外开销, 因而相较于 128 比特分组长度的版本, 256 比特分组长度的版本不是很适合在资源受限的低端微处理器上实现.

5.4 硬件实现

为了评估 TANGRAM 的硬件实现性能, 我们针对两类硬件实现平台进行评估, 即专门应用的集成电路 (ASIC) 和可编程逻辑阵列 (FPGA). 我们选取了两个常见的工艺库来评估 ASIC 硬件实现: UMC 130 nm 和学术界常用的开源工艺库 Nangate 45 nm. Xilinx 公司的 7 系 xc7s100fpga676-1FPGA 平台被用来评估 TANGRAM 在 FPGA 上的实现结果和性能.

我们准备了可用于 ECB 模式的 TANGRAM 三个版本的硬件实现. 除此之外, 为了说明 TANGRAM 算法硬件实现的灵活性, 我们同时提供了基于 CTR 模式的纯加密模块的轻量级实现和高速实现.

我们采用了多种优化方式来实现 TANGRAM 的 ECB 模式. 如图 8 所示, 其中加解密算法共享多个基本模块单元, 包括轮子密钥加、轮状态、密钥存储等. 对于 TANGRAM 128/256 版本, 因为密钥扩展算法采用了 Feistel 网络, 所以可以省去密钥逆向扩展模块. 为了节约轮子密钥的存储空间, 我们采取了在线生成轮密钥的方式. 对解密操作, 对于同一次 ECB 操作中的所有分组数据, 我们只在第一组数据到来之后, 正向运算出最末轮的轮密钥, 并存储在额外的存储模块 (Inv Key Storage) 中, 之后的解密操作通过在线执行密钥逆向扩展模块来在线生成.

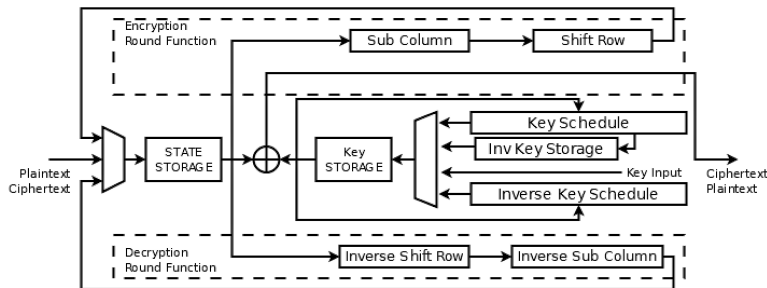


图 8 TANGRAM 的 ECB 模式的硬件结构

Figure 8 Hardware structure of TANGRAM in ECB mode

TANGRAM 的 ECB 模式的硬件实现在 UMC 130 nm 和 Nangate 45 nm 下进行测试, 结果如表 16 和表 17 所示. 其中运算周期涵盖全部 32 组数据所需的系统时钟数 (结果由 Modelsim 的功能仿真所验证). 我们可以看到解密比加密多花了一组加密操作的时钟周期, 这是由于我们需要生成最后一轮的子密钥用于反推, 但是这个操作只需要执行一次. 关键路径的时长是通过 Design Compiler 软件分析综合网表后, 通过分析时钟约束, 来选取最长延迟下 (maximum delay) 下的关键路径所得. 加解密速度根据第二轮材料说明中给出的计算方法得到. 表中的面积是 Design Compiler 中所给出的 total area 的面积. 经过查验对应工艺库的 PDK 说明文档, 本设计所采用的工艺库的面积单元为平方微米. 等效门 GE 是通过比较实现面积和该工艺下 NAND2 门所需面积的比值来计算, 在 UMC 130 nm 工艺库下, NAND2 所需要的面积是 4 um^2 , 而 Nangate 45 nm 下 NAND2 面积是 0.798 um^2 . 我们可以看到, 不同工艺节点对整体面积的影响非常大, 而等效门数基本上保持相似.

表 18 给出了 TANGRAM 的 ECB 模式在主流 FPGA 上的实现结果. 通过设定时钟约束, 然后综合加后端的布局布线, 对于三个版本的 TANGRAM 的 ECB 模式均能满足对应时钟约束的要求. 表 18 同时给出了在对应时钟约束下, TANGRAM 的 ECB 模式能够达到的加解密吞吐要求.

考虑到不同应用需求, 我们以 128/128 版本的 TANGRAM 为例, 针对不同的应用场景进行硬件实现

的优化. 实际应用中, 为了节省面积和高速并行化处理, 计数器模式 CTR 被广泛采用. 采用 CTR 模式, 硬件上只需要实现加密模块, 而不需要考虑解密模块, 能够有效的减少硬件开销. 同时 CTR 模式可以多个分组块并行计算, 非常有利于高速处理大数据量, 这一特性使得利用 GPU 和硬件实现来进行加速变得更加有效. 因此, 接下来对于不同应用场景的分析, 我们聚焦于 TANGRAM 的 CTR 模式实现.

表 16 TANGRAM 的 ECB 模式在 UMC 130 nm 工艺库下的自测结果 (全 32 组数据)
Table 16 Results of TANGRAM in UMC 130 nm technology library (ECB mode)

项目	128/128	128/256	256/256
运算周期	加密: 1440/解密: 1485	加密: 1632/解密: 1683	加密: 2656/解密: 2739
时钟约束	6.83 ns	7.73 ns	9.03 ns
加密速率	416 Mbps	324.7 Mbps	341 Mbps
解密速率	404 Mbps	314.8 Mbps	331.2 Mbps
面积	26 607 um ² (约为 6652 GE)	31 656 um ² (约为 7914 GE)	52 605 um ² (约为 13 152 GE)
加密吞面比	0.015 63 Mbps/um ²	0.010 26 Mbps/um ²	0.006 48 Mbps/um ²
解密吞面比	0.015 18 Mbps/um ²	0.009 94 Mbps/um ²	0.006 30 Mbps/um ²

表 17 TANGRAM 的 ECB 模式在 Nangate 45 nm 工艺库下的自测结果 (全 32 组数据)
Table 17 Results of TANGRAM in Nangate 45 nm technology library (ECB mode)

项目	128/128	128/256	256/256
运算周期	加密: 1440/解密: 1485	加密: 1632/解密: 1683	加密: 2656/解密: 2739
时钟约束	1.26 ns	1.51 ns	1.74 ns
加密速率	2257 Mbps	1662 Mbps	1773 Mbps
解密速率	2189 Mbps	1612 Mbps	1719 Mbps
面积	5025 um ² (约为 6297 GE)	6249 um ² (约为 7831 GE)	9975 um ² (约为 12 500 GE)
加密吞面比	0.449 Mbps/um ²	0.266 Mbps/um ²	0.178 Mbps/um ²
解密吞面比	0.436 Mbps/um ²	0.258 Mbps/um ²	0.172 Mbps/um ²

表 18 TANGRAM 的 ECB 模式在 Xilinxxc7s100fgga676-1FPGA 平台下的自测结果
Table 18 Results on Xilinxxc7s100fgga676-1FPGA platform in ECB mode

项目	128/128	128/256	256/256
所需查找表 LUTs	863	854	1526
所需寄存器 FFs	397	653	783
所需 BRAM 和 DSP	0	0	0
时钟约束	5 ns	4 ns	5 ns
加密速率	586.89 Mbps	627.45 Mbps	924.08 Mbps
解密速率	551.65 Mbps	608 Mbps	953.11 Mbps

考虑到 IoT 设备的广泛使用, 低面积实现往往是设计者对于轻量级应用的首要考虑因素. 我们这里实现了 Round-based TANGRAM. 因为上述说明, 我们这里采用 CTR 模式, 仅保留其加密部分. 表 19 中给出了 Round-based TANGRAM 的实现结果, 并与其它模式进行了对比. 我们需要强调的是, 由于 TANGRAM 的高度灵活性, 可以采用类似 RECTANGLE 的优化实现中串行实现的做法来进一步优化所需面积, 通过高度复用 AddRoundKey 模块和仅仅使用 1 个 S 盒来达到面积最优的串行实现.

另一个应用场景是服务器和数据中心. 这一类平台所需要处理的数据量往往非常巨大. 为了满足这一类应用场景, 我们给出了完全展开的全流水的版本. 这一版本中, 我们将所有轮的运算都实例化, 并在每轮之间加入寄存器来达到流水线实现的目的. 表 19 给出了全流水版本 TANGRAM 的硬件性能. 这里将数据列出只是为了表明对于极端应用场景下, 得益于 TANGRAM 算法的高灵活性, TANGRAM 往往不会是包含其的加密系统中的速度瓶颈.

这里我们希望说明, 高吞吐量实现和低延迟实现的需求往往并不一致. 高吞吐量实现要求单位时间内能够并行处理大的数据总量. 低延迟实现对数据加密开始到结束之间的延迟要求较高, 典型的应用有存储器加密. 一般通过完全展开轮函数, 并且在每一轮之间不插入寄存器这一方法来达到低延迟实现, 这种实现也被叫做单一时钟周期实现 (one-clock-cycle implementation).

表 19 多种应用场景下的 TANGRAM128/128 实现对比 (Nangate 45 nm)
Table 19 Performance of TANGRAM in multiple implementation mode (Nangate 45 nm)

对比项	ECB round based	CTR round based	CTR 全流水
面积	5025 um ² (约为 6297 GE)	2914.6 um ² (约为 3653 GE)	81 147 um ² (约为 101 688 GE)
最大系统频率	793.6 MHz	1.81 GHz	1.72 GHz
吞吐量	2.257 Gbps	5.148 Gbps	220.16 Gbps
功耗	747 uW	632 uW	400 860 uW
吞吐量/面积	0.449 Mbps/um ²	1.766 Mbps/um ²	2.713 Mbps/um ²
每比特能耗	3.3097×10^{-13} J/bit	1.2277×10^{-13} J/bit	18.21×10^{-13} J/bit

根据上面多个平台多个版本的 TANGRAM 硬件实现结果, 我们认为 TANGRAM 算法是一个高度灵活的、非常硬件实现友好型的分组密码. 得益于其规整和统一的轮函数和密钥扩展算法, 一方面针对低面积的硬件实现, 可以通过高度复用相关运算模块; 另一方面, 针对高吞吐量和低延迟的相关应用, 可以轻松实例化所有轮函数并行化来实现.

6 优点和创新点

TANGRAM 的主要优点和创新点如下:

(1) 设计简洁、易于扩展

TANGRAM 的整体结构为 SP 网络, 设计非常简洁: 对于 TANGRAM-128, 128 比特的加密状态用一个 4×32 的比特矩阵描述, S 层对每一列的 4 个比特做 S 盒替换, P 层对每一行的 32 个比特做左循环移位; 对于 TANGRAM-256, 256 比特的加密状态用一个 4×64 的比特矩阵描述, S 层对每一列的 4 个比特做 S 盒替换, P 层对每一行的 64 个比特做左循环移位.

TANGRAM 的 3 个版本基于同一框架和设计方法, 具有很好的可扩展性. 利用这种设计思路, 对于任意正整数 t , 可以类似地设计出分组长度为 $4t$ 比特的分组密码或者固定置换, 以满足不同安全需求和应用场景, 比如资源有限的轻量级分组密码、认证加密方案、密码杂凑函数等.

(2) 安全性分析深入

我们评估了 TANGRAM 抵抗差分、线性、积分、不可能差分以及相关密钥等分析方法的安全性. TANGRAM 128/128 的总轮数为 44 轮, 根据评估结果, 我们最多能攻击到 28 轮; 6 轮 TANGRAM-128 达到全扩散, 因此我们在 44 轮的基础上又增加了 6 轮, 将 50 轮作为 TANGRAM 128/256 的总轮数; TANGRAM 256/256 的总轮数为 82 轮, 我们估计最多能攻击到 56 轮.

我们认为: 以当前分组密码分析方法的发展现状作为参考, 我们为 TANGRAM 的 3 个版本预留的冗余都足够保证其抵抗数学类攻击的安全性.

(3) 软件和硬件实现均表现很好

TANGRAM 的主要设计思想是采用比特切片方法来设计适合多个软硬件平台的分组密码. 我们在多个不同平台上实现了 TANGRAM, 结果表明 TANGRAM 的软件和硬件实现均表现很好, 因此可以灵活

地适用于多种应用场景.

软件实现方面: (1) 在 X64 平台上, 根据我们的测试结果, TANGRAM 128/128、TANGRAM 128/256、TANGRAM 256/256 的基本软件实现的加密速度分别是 23.4、26.4 和 22.0 cycles/byte; 在需要更为高速的应用场景, 采用 CTR 模式, 利用 AVX 指令集, TANGRAM 128/128、TANGRAM 128/256、TANGRAM 256/256 的加密速度可分别达到 4.7、5.2 和 8.9 cycles/byte. (2) 在 32 位 ARM Cortex-M3 平台上, 可以非常方便地实现 TANGRAM. 根据我们的测试结果, TANGRAM 128/128、TANGRAM 128/256、TANGRAM 256/256 的加密速度分别是 176.1、219.0 和 518.9 cycles/byte. (3) 在 8 位 AVR 平台上, 由我们测试结果可以看出, 对 TANGRAM-128, 在所考虑的第一种应用情境中 (采用 CBC 模式加解密数据且包含密钥扩展算法), 在保持适中的运行时间的同时只需要极小的代码量; 在所考虑的第二种应用情境中 (采用 CTR 模式加密数据, 轮子密钥直接写入程序中), 只需要可忽略的随机存储内存.

硬件实现方面, 我们选取了 UMC 130 nm 和 Nangate 45 nm 两个常见的工艺库来评估 ASIC 实现、以及 Xilinx 公司的 7 系 xc7s100fpga676-1 平台来评估 FPGA 实现. 我们给出了 TANGRAM 三个版本在 ECB 模式下的硬件实现; 除此之外, 为了方便说明 TANGRAM 硬件实现的灵活性, 我们同时提供了 CTR 模式下的纯加密模块的轻量级实现和高速实现. 通过分析多个平台多个版本的 TANGRAM 硬件实现结果, 我们认为 TANGRAM 是一个硬件实现友好型的分组密码, 同时具有极高的灵活性和可扩展性. 得益于其规整统一的轮函数和密钥扩展函数, 可以通过高度复用相关运算模块来满足低面积硬件实现的需求, 也可以通过实例化所有轮函数并行化来满足高吞吐量和低延迟的相关应用. 设计 TANGRAM 的硬件实现简单便捷, 仅需要较低的工程经验, 非常适合快速修改整合到任意需要分组密码的系统, 以提供满足针对性需求的硬件实现方案.

(4) 防护侧信道代价低

得益于简洁的设计和比特切片方法, TANGRAM 防护侧信道攻击的代价很低. 以 Threshold 防护为例, TANGRAM 的 S 盒是其中唯一复杂的部分, 根据 Bilgin 博士等人发表于 CHES 2012 的论文^[17], TANGRAM 的 S 盒属于第 266 类, 仅需要 3 个 shares, 并且在每一个 share 中, 一阶防护仅需一对 G 和 F 函数; 此外, TANGRAM 基于比特切片方法设计, 相比于查表实现, 比特切片的软件实现能够抵抗时间缓存攻击.

(5) 设计准则公开透明

TANGRAM 的主要设计思想是: 采用比特切片方法设计适合多个软硬件平台的分组密码. TANGRAM 采用了 SP 网络, S 层是 32(或 64) 个 4×4 的 S 盒的并置, P 层是一个比特置换. 第3节我们详细给出了 TANGRAM 的整体设计思想和各个模块的选取准则. 可以说, TANGRAM 的设计, 没有晦涩之处, 都是基于我们对其模块的密码性质以及整体密码算法的安全性的充分理解和把握的基础之上.

(6) 对模块的仔细选择

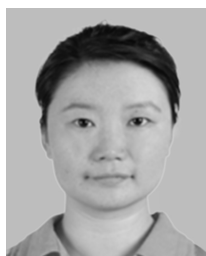
在设计 TANGRAM 的过程中, 我们细致地挑选目标 S 盒, 对它的两个密码指标做了特殊限制, 以提升它抵抗差分/线性密码分析的性价比, 详见第3.3节 TANGRAM 的 S 盒设计以及文献 [18]; P 置换的设计也非常重要, TANGRAM 的 P 置换仅包含 3 个 32(或 64) 比特字上的循环移位, 无论在硬件还是软件上都可以容易、高效地实现, 这 3 个循环移位参数需要仔细选择, 以提升算法的整体安全强度, 详见第3.4节中的 TANGRAM 的扩散层设计; 将 TANGRAM 的 S 盒和 P 置换组合在一起, 所构成的整体密码算法具有很好的抵抗差分/线性密码分析的能力. 上述因素综合在一起, TANGRAM 才得以在安全性和实现性能之间达到了很好的平衡.

References

- [1] BIHAM E. A fast new DES implementation in software[C]. In: Fast Software Encryption—FSE '97. Springer Berlin Heidelberg, 1997: 260–272. [DOI 10.1007/BFb0052352]
- [2] LEANDER G, POSCHMANN A. On the classification of 4 bit S-boxes[C]. In: Arithmetic of Finite Fields—WAIFI 2007. Springer Berlin Heidelberg, 2007: 159–176. [DOI 10.1007/978-3-540-73074-3_13]
- [3] ZHANG W, BAO Z, RIJMEN V, et al. A new classification of 4-bit optimal S-boxes and its application to PRESENT, RECTANGLE and SPONGENT[C]. In: Fast Software Encryption—FSE 2015. Springer Berlin Heidelberg, 2015: 494–515. [DOI: 10.1007/978-3-662-48116-5_24]

- [4] BIHAM E, SHAMIR A. Differential cryptanalysis of DES-like cryptosystems[J]. Journal of Cryptology, 1991, 4(1): 3–72. [DOI: 10.1007/BF00630563]
- [5] BIHAM E, SHAMIR A. Differential Cryptanalysis of the Data Encryption Standard[M]. Springer New York, NY, USA. 1993: 1–188. [DOI: 10.1007/978-1-4613-9314-6]
- [6] BAO Z, ZHANG W, LIN D. Speeding up the search algorithm for the best differential and best linear trails[C]. In: Information Security and Cryptology—INSCRYPT 2014. Springer Cham, 2015: 259–285. [DOI: 10.1007/978-3-319-16745-9_15]
- [7] MATSUI M. The first experimental cryptanalysis of the data encryption standard[C]. In: Advances in Cryptology—CRYPTO '94. Springer Berlin Heidelberg, 1994: 1–11. [DOI: 10.1007/3-540-48658-5_1]
- [8] MATSUI M. Linear cryptanalysis method for DES cipher[C]. In: Advances in Cryptology—EUROCRYPT '93. Springer Berlin Heidelberg, 1994: 386–397. [DOI: 10.1007/3-540-48285-7_33]
- [9] BIHAM E, BIRYUKOV A, SHAMIR A. Cryptanalysis of Skipjack reduced to 31 rounds using impossible differentials[C]. In: Advances in Cryptology—EUROCRYPT '99. Springer Berlin Heidelberg, 1999: 12–23. [DOI: 10.1007/3-540-48910-X_2]
- [10] DAEMEN J, KNUDSEN L, RIJMEN V. The block cipher Square[C]. In: Fast Software Encryption—FSE '97. Springer Berlin Heidelberg, 1997: 149–165. [DOI: 10.1007/BFb0052343]
- [11] KNUDSEN L, WAGNER D. Integral cryptanalysis[C]. In: Fast Software Encryption—FSE 2002. Springer Berlin Heidelberg, 2002: 112–127. [DOI: 10.1007/3-540-45661-9_9]
- [12] TODO Y. Structural evaluation by generalized integral property[C]. In: Advances in Cryptology—EUROCRYPT 2015, Part I. Springer Berlin Heidelberg, 2015: 287–314. [DOI: 10.1007/978-3-662-46800-5_12]
- [13] XIANG Z, ZHANG W, BAO Z, et al. Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers[C]. In: Advances in Cryptology—ASIACRYPT 2016, Part I. Springer Berlin Heidelberg, 2016: 648–678. [DOI: 10.1007/978-3-662-53887-6_24]
- [14] BIRYUKOV A, WAGNER D. Slide attacks[C]. In: Fast Software Encryption—FSE '99. Springer Berlin Heidelberg, 1999: 245–259. [DOI: 10.1007/3-540-48519-8_18]
- [15] BIRYUKOV A, WAGNER D. Advanced slide attacks[C]. In: Advances in Cryptology—EUROCRYPT 2000. Springer Berlin Heidelberg, 2000: 589–606. [DOI: 10.1007/3-540-45539-6_41]
- [16] BIHAM E. New types of cryptanalytic attacks using related keys[C]. In: Advances in Cryptology—EUROCRYPT '93. Springer Berlin Heidelberg, 1994: 398–409. [DOI: 10.1007/3-540-48285-7_34]
- [17] BILGEN B, NIKOVA S, NIKOV V, et al. Threshold implementations of all 3×3 and 4×4 S-boxes[C]. In: Cryptographic Hardware and Embedded Systems—CHES 2012. Springer Berlin Heidelberg, 2012: 76–91. [DOI: 10.1007/978-3-642-33027-8_5]
- [18] ZHANG W, BAO Z, LIN D, et al. Rectangle: A bit-slice lightweight block cipher suitable for multiple platforms[J]. SCIENCE CHINA Information Sciences, 2015, 58(12): 1–15. [DOI: 10.1007/s11432-015-5459-7]

作者信息



张文涛 (1977–), 陕西丹凤人, 博士, 研究员. 主要研究领域为对称密码的安全性分析与设计.
zhangwentao@iie.ac.cn



季福磊 (1993–), 山东青岛人, 博士. 主要研究领域为对称密码的安全性分析与设计.
jifulei@iie.ac.cn



丁天佑 (1993–), 上海人, 博士. 主要研究领域为对称密码的安全性分析与设计.
dingtianyou@iie.ac.cn



杨博翰 (1986–), 陕西咸阳人, 博士, 博士后. 主要研究领域为硬件安全与密码芯片.
bohanyang@tsinghua.edu.cn



赵雪峰 (1994–), 山西阳泉人, 博士. 主要研究领域为对称密码的安全性分析与设计.
zhaoxuefeng@iie.ac.cn



向泽军 (1990–), 湖北孝感人, 博士, 讲师. 主要研究对称密码的安全性分析与设计.
xiangzejun@hubu.edu.cn



包珍珍 (1988–), 北京房山人, 博士, 主要研究领域为对称密码的分析, 设计与实现.
15201279013@163.com



刘雷波 (1975–), 重庆人, 博士, 教授. 主要研究领域为硬件安全与密码芯片.
liulb@tsinghua.edu.cn

附录 TANGRAM 的比特切片实现描述

为方便描述, 我们对 TANGRAM-128 和 TANGRAM-256 的密码状态进行统一的定义, 在 TANGRAM-128 和 TANGRAM-256 中, 这些符号分别代表不同长度的比特子块.

TANGRAM-128 对于图 2(b) 所示的一个密码状态 W , 将它表示为 4 个 32 比特子块的连接, 即 $W = A_3 \| A_2 \| A_1 \| A_0$, 其中, A_i 是 W 的第 i 行, $A_i = a_{i,31} \| \cdots \| a_{i,1} \| a_{i,0}$, $i = 0, 1, 2, 3$.

TANGRAM-256 对于图 3(b) 所示的一个密码状态 W , 将它表示为 4 个 64 比特子块的连接, 即 $W = A_3 \| A_2 \| A_1 \| A_0$, 其中, A_i 是 W 的第 i 行, $A_i = a_{i,63} \| \cdots \| a_{i,1} \| a_{i,0}$, $i = 0, 1, 2, 3$.

(1) 加密轮函数

轮密钥加 AddRoundKey: 设轮函数的输入为 $I_3 \| I_2 \| I_1 \| I_0$, 轮子密钥为 $SK_3 \| SK_2 \| SK_1 \| SK_0$, 两者异或后的结果记为 $A_3 \| A_2 \| A_1 \| A_0$, 则有:

$$A_0 := I_0 \oplus SK_0, A_1 := I_1 \oplus SK_1, A_2 := I_2 \oplus SK_2, A_3 := I_3 \oplus SK_3$$

列替换 SubColumn: 用 $A_3 \| A_2 \| A_1 \| A_0$ 和 $B_3 \| B_2 \| B_1 \| B_0$ 分别表示 SubColumn 的输入和输出, 则有:

$$\begin{aligned} T_1 &:= A_0 \oplus A_2, T_2 := A_0 \& A_1, T_3 := A_3 \oplus T_2, B_2 := A_2 \oplus T_3, T_5 := A_1 \oplus A_2, T_6 := T_1 \& T_3 \\ B_0 &:= T_5 \oplus T_6, T_8 := A_1 | A_3, T_9 := T_1 \oplus T_8, B_3 := \sim T_9, T_{11} := T_5 \& T_9, B_1 := T_3 \oplus T_{11} \end{aligned}$$

其中, “&”, “|”, “ $\sim$ ”, “ $\oplus$ ” 分别表示按位与、或、非和异或. T_i 是 32 比特字 (对应 TANGRAM-128) 或 64 比特字 (对应 TANGRAM-256) 的临时变量, $i = 1, 2, 3, 5, 6, 8, 9, 11$.

行移位 ShiftRow: 用 $B_3 \| B_2 \| B_1 \| B_0$ 和 $C_3 \| C_2 \| C_1 \| C_0$ 分别表示 ShiftRow 的输入和输出.

$$\text{TANGRAM-128: } C_1 := B_1 \lll 1, C_2 := B_2 \lll 8, C_3 := B_3 \lll 11$$

$$\text{TANGRAM-256: } C_1 := B_1 \lll 1, C_2 := B_2 \lll 8, C_3 := B_3 \lll 41$$

其中, $A \lll x$ 表示对 32 比特字 (对应 TANGRAM-128) 或 64 比特字 (对应 TANGRAM-256) 左循环移动 x 位.

(2) 解密轮函数

轮密钥加 **AddRoundKey**: 与加密轮函数中的 **AddRoundKey** 相同.

列替换的逆变换 **InverseSubColumn**: 用 $B_3\|B_2\|B_1\|B_0$ 和 $A_3\|A_2\|A_1\|A_0$ 分别表示 **InverseSubColumn** 的输入和输出, 则有:

$$\begin{aligned} T_1 &:= \sim A_3, T_2 := A_0 \& T_1, T_3 := A_1 \oplus T_2, B_2 := A_2 \oplus T_3, T_5 := A_0 \oplus A_2, T_6 := T_1 \& T_3, \\ B_1 &:= T_5 \oplus T_6, T_8 := A_0 \oplus T_1, T_9 := T_3 \& T_5, B_0 := T_8 \oplus T_9, T_{11} := B_1 \& B_0, B_3 := T_3 \oplus T_{11} \end{aligned}$$

其中, T_i 是 32 比特字 (对应 TANGRAM-128) 或 64 比特字 (对应 TANGRAM-256) 的临时变量, $i = 1, 2, 3, 5, 6, 8, 9, 11$.

行移位的逆变换 **InverseShiftRow**: 用 $C_3\|C_2\|C_1\|C_0$ 和 $B_3\|B_2\|B_1\|B_0$ 分别表示 **InverseShiftRow** 的输入和输出.

$$\text{TANGRAM-128: } B_1 := C_1 \ggg 1, B_2 := C_2 \ggg 8, B_3 = C_3 \ggg 11$$

$$\text{TANGRAM-256: } B_1 := C_1 \ggg 1, B_2 := C_2 \ggg 8, B_3 = C_3 \ggg 41$$

其中, $A \ggg x$ 表示对 32 比特字 (对应 TANGRAM-128) 或 64 比特字 (对应 TANGRAM-256) 右循环移动 x 位.